



Dijkstra and Bellman–Ford

1 Shortest Paths

Recall that a weighted graph G is a graph equipped with a weight function $w : E \rightarrow \mathbb{R}$, where the weight of a path is defined as the sum of the weights of the edges along the path. The shortest path between two vertices is a path that connects them and is also minimal in its weight. One native application of this formulation is to model routes between destinations, where edges correspond to roads and vertices to mid-stations.

Two main algorithms are used to solve the shortest path problem. The first is Dijkstra, which is required only when the weight function is **non-negative** and runs in time $\Theta(|E| \log |V|)$ ¹. The second is Bellman-Ford, which handles **negative** weights, yet it does so at a higher running time cost of $\Theta(|E| \cdot |V|)$.

Initially the distances from the source are initialized to ∞ and along the running they are decreased closer to their real value by application of relaxations.

Claim 1. *Given a weighted (can be directed) graph G with weights $w : E \rightarrow \mathbb{R}$, let $v_0, v_1, \dots, v_k$ be a shortest path from v_0 to v_k then for any $0 \leq i < j \leq k$ the path $v_i, v_{i+1}, \dots, v_{j-1}, v_j$ is a shortest path from v_i to v_j .*

Proof. Assume by contradiction that there is a path $v_i, u_{i+1}, \dots, u_{j-1}, v_j$ that has a lower weight than $v_i, v_{i+1}, \dots, v_{j-1}, v_j$ then:

$$\begin{aligned} w(v_0, v_1, \dots, v_k) &= w(v_0, v_1, \dots, v_i) + w(v_i, v_{i+1}, \dots, v_j) + w(v_j, v_{j+1}, \dots, v_k) \\ &> w(v_0, v_1, \dots, v_i) + w(v_i, u_{i+1}, \dots, u_{j-1}, v_j) + w(v_j, v_{j+1}, \dots, v_k) \\ &= w(v_0, v_1, \dots, v_i, u_{i+1}, \dots, u_{j-1}, v_j, \dots, v_k) \end{aligned}$$

□

Relaxation.

```
1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3   Update  $(Q, d[v])$ 
4    $\pi[v] \leftarrow u$ 
5 end
```

Algorithm 1: Relaxation.

Shortest-paths Properties

¹Can be improved to $|E| + |V| \log |V|$, but that is beyond the scope of the course.

Claim 2 (Triangle inequality). *For all edges $(u, v) \in E$ we have:*

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

Claim 3 (Upper-bound property). *Suppose that for every v $d[v]$ is initialized such that $d[v] > \delta(s, v)$ then any application of any sequence relaxation steps $d[v] > \delta(s, v)$*

Claim 4 (No-path property). *Like above for $\delta(s, v) = \infty$*

Claim 5 (Convergence property). *$s \rightsquigarrow u \rightarrow v$, last stage.*

Claim 6 (Path-relaxation property). *Let $s = v_0, v_1, v_2, \dots, v_k$ be a shortest path from s to v_k . After an application of a sequence relaxation steps that includes, in order, $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ we have that $d[v_k] = \delta(s, v_k)$.*

Proof. We show by induction on i that after the relaxation of (v_{i-1}, v_i) we have $d[v_i] = \delta(s, v_i)$.

1. Base.
2. Assumption.
3. Step.

□

2 Dijkstra

Dijkstra can be thought of as BFS where the FIFO queue is replaced by a min-heap. Each time we extract the closest unvisited vertex, we scan its edges and check for each of its neighbors if getting to it through their edge defines a shorter path than what is currently known. If so, we update the heap. Algorithm 2 gives an implementation.

What's wrong with negative weights? Consider the following graph,

3 Bellman-Ford

Detecting a Negative Cycle. Suppose that $v_1, v_1, \dots, v_k = v_1$ is a negative cycle. This means that:

$$\sum_{i=2}^k w(v_{i-1}, v_i) < 0$$

Assume that the algorithm doesn't detect the cycle, namely the condition at Line 14 is not satisfied for any of the edges. Then:

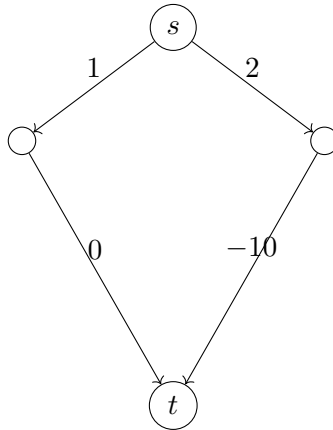


Figure 5.1: Example of a binary tree with 12 vertices. In blue, one of the longest paths from the root to one of the leaves, which is of length 4.

$$\begin{aligned}
 & \forall v_i \ d[v] < d[u] + w(u, v) \\
 \Rightarrow & \sum_{i=2}^k d[v_i] < \sum_{i=2}^k d[v_{i-1}] + \sum_{i=2}^k w(v_{i-1}, v_i) \\
 & = \sum_{i=2}^k d[v_i] + \sum_{i=2}^k w(v_{i-1}, v_i) \\
 & 0 \leq \sum_{i=2}^k w(v_{i-1}, v_i)
 \end{aligned}$$

Correctness. at the i iteration, we apply the relaxation (v_{i-1}, v_i) after we apply it over $(v_1, v_2), (v_2, v_3), \dots, (v_{i-2}, v_{i-1})$

```

1 for  $v \in V$  do
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow \text{null}$ 
4 end
5  $d[s] \leftarrow 0$ 
6  $Q \leftarrow \text{Heap}(d[v_1], d[v_2], \dots, d[v_n])$ 
7 while  $Q$  is not empty do
8    $u \leftarrow \text{extract-min}(Q)$ 
9   for  $(u, v) \in E$  do
10    if  $d[v] > d[u] + w(u, v)$  then
11       $d[v] \leftarrow d[u] + w(u, v)$ 
12       $\text{Update}(Q, d[v])$ 
13       $\pi[v] \leftarrow u$ 
14    end
15  end
16 end

```

Algorithm 2: Dijkstra.

```

1 for  $v \in V$  do
2    $d[v] \leftarrow \infty$ 
3    $\pi[v] \leftarrow \text{null}$ 
4 end
5  $d[s] \leftarrow 0$ 
6 for  $i \in [n]$  do
7   for  $(u, v) \in E$  do
8     if  $d[v] > d[u] + w(u, v)$  then
9        $d[v] \leftarrow d[u] + w(u, v)$ 
10       $\pi[v] \leftarrow u$ 
11    end
12  end
13 end
14 for  $(u, v) \in E$  do
15   if  $d[v] > d[u] + w(u, v)$  then
16     return Negative Cycle
17   end
18 end

```

Algorithm 3: Bellman–Ford.